Documentation technique – AP C#

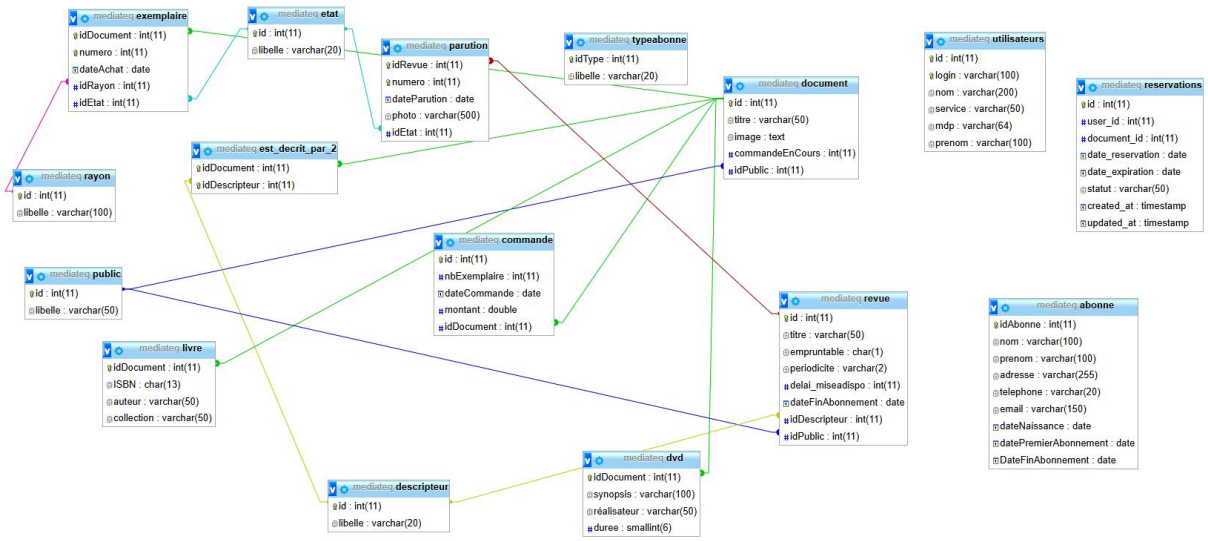
Documentation technique Médiateq

Session 2026

CARTY JAMAR
AP Mediateq C#

Architecture de la base de donner

Concepteur de la base de donner Mediateq



Structure de la base de donner Mediateq

	Table	Action	Lignes	Type	Interclassement	Taille	Perte
☐	abonne	Afficher Structure Rechercher Insérer Vider Supprimer	4	InnoDB	utf8mb4_unicode_ci	16 Kio	-
☐	commande	Afficher Structure Rechercher Insérer Vider Supprimer	0	InnoDB	utf8mb4_general_ci	32 Kio	-
☐	descripteur	Afficher Structure Rechercher Insérer Vider Supprimer	19	InnoDB	utf8mb4_general_ci	16 Kio	-
☐	document	Afficher Structure Rechercher Insérer Vider Supprimer	26	InnoDB	utf8mb4_general_ci	32 Kio	-
☐	dvd	Afficher Structure Rechercher Insérer Vider Supprimer	0	InnoDB	utf8mb4_general_ci	16 Kio	-
☐	est_decrit_par_2	Afficher Structure Rechercher Insérer Vider Supprimer	4	InnoDB	utf8mb4_general_ci	32 Kio	-
☐	etat	Afficher Structure Rechercher Insérer Vider Supprimer	4	InnoDB	utf8mb4_general_ci	16 Kio	-
☐	exemplaire	Afficher Structure Rechercher Insérer Vider Supprimer	32	InnoDB	utf8mb4_general_ci	48 Kio	-
☐	livre	Afficher Structure Rechercher Insérer Vider Supprimer	25	InnoDB	utf8mb4_general_ci	16 Kio	-
☐	parution	Afficher Structure Rechercher Insérer Vider Supprimer	46	InnoDB	utf8mb4_general_ci	32 Kio	-
☐	public	Afficher Structure Rechercher Insérer Vider Supprimer	4	InnoDB	utf8mb4_general_ci	16 Kio	-
☐	rayon	Afficher Structure Rechercher Insérer Vider Supprimer	5	InnoDB	utf8mb4_general_ci	16 Kio	-
☐	reservations	Afficher Structure Rechercher Insérer Vider Supprimer	1	InnoDB	utf8mb4_unicode_ci	16 Kio	-
☐	revue	Afficher Structure Rechercher Insérer Vider Supprimer	11	InnoDB	utf8mb4_general_ci	48 Kio	-
☐	typeabonne	Afficher Structure Rechercher Insérer Vider Supprimer	3	InnoDB	utf8mb4_unicode_ci	16 Kio	-
☐	utilisateurs	Afficher Structure Rechercher Insérer Vider Supprimer	2	InnoDB	utf8mb4_general_ci	32 Kio	-
	16 tables	Somme	186	InnoDB	utf8mb4_unicode_ci	400 Kio	0 o

Table livre :

```
-- Structure de la table `livre`
--

CREATE TABLE `livre` (
  `idDocument` int(11) NOT NULL,
  `ISBN` char(13) DEFAULT NULL,
  `auteur` varchar(50) DEFAULT NULL,
  `collection` varchar(50) DEFAULT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

--
```

Table Rayon :

```sql
-- Structure de la table `rayon`
--

CREATE TABLE `rayon` (
  `id` int(11) NOT NULL,
  `libelle` varchar(100) DEFAULT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

--
-- Déchargement des données de la table `rayon`
--

INSERT INTO `rayon` (`id`, `libelle`) VALUES
(1, 'Romans'),
(2, 'Science-fiction'),
(3, 'Policier'),
(4, 'Jeunesse'),
(5, 'Biographie');
```

Table revue :

```sql
-- Structure de la table `revue`
--

CREATE TABLE `revue` (
  `id` int(11) NOT NULL,
  `titre` varchar(50) NOT NULL,
  `empruntable` char(1) DEFAULT NULL,
  `periodicite` varchar(2) DEFAULT NULL,
  `delai_miseadispo` int(11) DEFAULT NULL,
  `dateFinAbonnement` date DEFAULT NULL,
  `idDescripteur` int(11) NOT NULL,
  `idPublic` int(11) NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

--
```

Technologie utiliser

Prérequis

Système d'exploitation : Windows 10 ou Windows 11 compatible

- Le .NET Framework 4.7.2 doit être installé
- Un **serveur MySQL accessible**, contenant la base de données requise
- Des **droits de lecture et d'écriture** sur le dossier du projet pour permettre l'exécution si nécessaire

- **Visual Studio 2026 (optionnel)** pour le lancement en mode debug
- **Fichier de configuration important**
- bdd\DAOFactory.cs : paramètres de connexion à la base de données

Lancement du projet

- Récupérer l'archive ou cloner le dépôt.
- Ouvrir la solution dans **Visual Studio 2022**.
- En environnement de développement, restaurer les packages NuGet via le **Gestionnaire de packages NuGet**.
- Vérifier et modifier si nécessaire la chaîne de connexion à la base MySQL dans bdd\DAOFactory.cs.
- Compiler la solution via **Build > Build Solution**.
- Lancer l'application :
- Depuis Visual Studio : cliquer sur **Démarrer**.
- Ou double-cliquer sur l'exécutable dans le dossier bin\Debug.

Identification

- Sur l'écran d'identification, renseigner le pseudo et le mot de passe.
- Les mots de passe sont stockés en base sous forme hachée **(BCrypt)**. Pour un nouvel utilisateur créé manuellement, insérer un hash BCrypt valide.
- Messages d'erreur courants :
- "Veuillez remplir tous les champs" : un ou plusieurs champs sont vides.
- "Erreur dans l'identification" : identifiants incorrects.
- "Connexion base de données impossible" : la base de données est inaccessible.

Collection liste variable global

```
//variables globales pour stocker les collections d'objets chargées en mémoire
 static List<Categorie> lesCatégories;
 static List<Descripteur> lesDescripteurs;
 static List<Revue> lesRevues;
 static List<Livre> lesLivres;
 static List<Exemplaire> lesExemplaires;
 static List<Abonne> lesAbonnes;
 static List<Utilisateurs> lesUtilisateurs;
 static List<Rayon> lesRayons;
 static List<Document> lesDocuments;
 static List<DVD> lesDVD;
 static List<Etat> lesEtat;
```

Classe métier

Présentation de la classe utilisée, avec la description de son constructeur et des méthodes d'accès (getters et setters) permettant de manipuler ses attributs.

Classe Abonner

```
18 références
internal class Abonne
{
    private int id;
    private string nom;
    private string prenom;
    private string adresse;
    private string telephone;
    private string email;
    public DateTime datePremierAbonnement;
    public DateTime dateFinAbonnement;
    private DateTime dateNaissance;


    4 références
    public Abonne(int id, string nom, string prenom, string adresse, string telephone, string email, DateTime dateNaissance,
    DateTime datePremierAbonnement, DateTime dateFinAbonnement)
    {
        this.id = id;
        this.nom = nom;
        this.prenom = prenom;
        this.adresse = adresse;
        this.telephone = telephone;
        this.email = email;
        this.dateNaissance = dateNaissance;
        this.datePremierAbonnement = datePremierAbonnement;
        this.dateFinAbonnement = dateFinAbonnement;

    }


    2 références
    public int idAbonne { get => id; set => id = value; }

    3 références
    public string Nom { get => nom; set => nom = value; }

    3 références
    public string Prenom { get => prenom; set => prenom = value; }

    3 références
    public string Adresse { get => adresse; set => adresse = value; }

    3 références
    public string Telephone { get => telephone; set => telephone = value; }

    3 références
    public string Email { get => email; set => email = value; }

    3 références
    public DateTime DateNaissance { get => dateNaissance; set => dateNaissance = value; }

    3 références
    public DateTime DatePremierAbonnement { get => datePremierAbonnement; set => datePremierAbonnement = value; }

    3 références
    public DateTime DateFinAbonnement { get => dateFinAbonnement; set => dateFinAbonnement = value; }


}
```

Classe catégorie

```
namespace Mediateq_AP_SIO2.metier
{
    20 références
    class Categorie
    {
        private int id;
        private string libelle;

        3 références
        public Categorie(int id, string libelle)
        {
            this.id = id;
            this.libelle = libelle;
        }

        0 références
        public int Id { get => id; set => id = value; }
        2 références
        public string Libelle { get => libelle; set => libelle = value; }
    }
}
```

Classe Descripteur

```
namespace Mediateq_AP_SIO2.metier
{
    21 références
    class Descripteur
    {
        private int id;
        private string libelle;

        3 références
        public Descripteur(int id, string libelle)
        {
            this.id = id;
            this.libelle = libelle;
        }

        2 références
        public int Id { get => id; set => id = value; }
        0 références
        public string Libelle { get => libelle; set => libelle = value; }
    }


}
```

Classe document

```
16 références
class Document
{
    private int idDoc;
    private string titre;
    private string image;
    private Categorie laCategorie;
    private List<Descripteur> lesDescripteurs;

    3 références
    public Document(int unId, string unTitre, string uneImage)
    {
        idDoc = unId;
        titre = unTitre;
        image = uneImage;
    }


    9 références
    public int IdDoc { get => idDoc; set => idDoc = value; }
    7 références
    public string Titre { get => titre; set => titre = value; }
    1 référence
    public string Image { get => image; set => image = value; }
    4 références
    public Categorie LaCategorie { get => laCategorie; set => laCategorie = value; }
    1 référence
    internal List<Descripteur> LesDescripteurs { get => lesDescripteurs; set => lesDescripteurs = value; }
}


}
```

Classe DVD

```
internal class DVD  : Document
{
    private string duree;
    private string realisateurs;
    private string synopsis;

    0 références
    public DVD(int idDoc, string titre, string image, Categorie laCategorie, List<Descripteur> lesDescripteurs, string duree,
    string realisateurs, string synopsis): base(idDoc, titre, image)
    {
        this.duree = duree;
        this.realisateurs = realisateurs;
        this.synopsis = synopsis;
    }

    0 références
    public string Duree { get => duree; set => duree = value; }

    0 références
    public string Realisateurs { get => realisateurs; set => realisateurs = value; }

    0 références
    public string Synopsis { get => synopsis; set => synopsis = value; }

    0 références
    public Categorie LaCategorie { get => base.LaCategorie; set => base.LaCategorie = value; }


}
}
```

Classe Etat

```
internal class Etat
{
    private int id;
    private string libelle;


    2 références
    public Etat(int id, string libelle)
    {
        this.id = id;
        this.libelle = libelle;
    }

    3 références
    public int ID { get => id; set => id = value; }
    3 références
    public string Libelle { get => libelle; set => libelle = value; }

}
```

Classe exemplaire

```
internal class Exemplaire
{
    private int idDocument;
    private int numero;
    private DateTime dateAchat;
    private int idRayon;
    private int idEtat;



4 références
public Exemplaire(int idDocument, int numero, DateTime dateAchat, int idRayon, int idEtat)
    {
        this.idDocument = idDocument;
        this.numero = numero;
        this.dateAchat = dateAchat;
        this.idRayon = idRayon;
        this.idEtat = idEtat;
    }


    7 références
    public int IdDocument { get => idDocument; set => idDocument = value; }
    6 références
    public int Numero { get => numero; set => numero = value; }

    5 références
    public DateTime DateAchat { get => dateAchat; set => dateAchat = value; }

    6 références
    public int IdRayon { get => idRayon; set => idRayon = value; }

    5 références
    public int IdEtat { get => idEtat; set => idEtat = value; }



}
```

Classe Livre

```
class Livre : Document
{
    private string ISBN;
    private string auteur;
    private string laCollection;


    1 référence
    public Livre(int unId, string unTitre, string unISBN, string unAuteur, string uneCollection,string uneImage) : base(unId, unTitre, uneImage)
    {
        ISBN1 = unISBN;
        Auteur = unAuteur;
        LaCollection = uneCollection;
    }

    3 références
    public string ISBN1 { get => ISBN; set => ISBN = value; }
    3 références
    public string Auteur { get => auteur; set => auteur = value; }
    3 références
    public string LaCollection { get => laCollection; set => laCollection = value; }
}
}
```

Classe parution

```
    class Parution
    {
        private int numero;
        private DateTime dateParution;
        private string photo;
        private Revue laRevue;

        1 référence
        public Parution(int numero, DateTime dateParution, string photo, Revue uneRevue)
        {
            this.numero = numero;
            this.dateParution = dateParution;
            this.photo = photo;
            this.laRevue = uneRevue;
        }

        1 référence
        public int Numero { get => numero; set => numero = value; }
        1 référence
        public DateTime DateParution { get => dateParution; set => dateParution = value; }
        1 référence
        public string Photo { get => photo; set => photo = value; }
        0 références
        public Revue LaRevue { get => laRevue; set => laRevue = value; }
    }
}
```

Classe Rayon

```
16 références
internal class Rayon
{
    private  int idRayon;
    private string libelle;



    2 références
    public Rayon(int idRayon, string libelle)
    {
        this.idRayon = idRayon;
        this.libelle = libelle;
    }



    5 références
    public int IdRayon { get => idRayon; set => idRayon = value; }

    3 références
    public string Libelle { get => libelle; set => libelle = value; }


    0 références
    public override string ToString() => libelle;



}
}
```

Classe Revue

```
17 références
class Revue
{
    private int id;
    private string titre;
    private char empruntable;
    private string periodicite;
    private DateTime dateFinAbonnement;
    private int delaiMiseADispo;
    private Descripteur leDescripteur;
    private Categorie laCategorie;

    1 référence
    public Revue(int id, string titre, char empruntable, string periodicite, DateTime dateFinAbonnement, int delaiMiseADispo)
    {
        this.id = id;
        this.titre = titre;
        this.empruntable = empruntable;
        this.periodicite = periodicite;
        this.dateFinAbonnement = dateFinAbonnement;
        this.delaiMiseADispo = delaiMiseADispo;
    }

    4 références
    public int Id { get => id; set => id = value; }
    1 référence
    public string Titre { get => titre; set => titre = value; }
    1 référence
    public char Empruntable { get => empruntable; set => empruntable = value; }
    0 références
    public string Periodicite { get => periodicite; set => periodicite = value; }
    1 référence
    public DateTime DateFinAbonnement { get => dateFinAbonnement; set => dateFinAbonnement = value; }
    1 référence
    public int DelaiMiseADispo { get => delaiMiseADispo; set => delaiMiseADispo = value; }
    2 références
    internal Descripteur LeDescripteur { get => leDescripteur; set => leDescripteur = value; }
    2 références
    internal Categorie LaCategorie { get => laCategorie; set => laCategorie = value; }
}
}
```

Classe utilisation

```
13 références
internal class Utilisateurs
{
    private int idUser;
    private string nom;
    private string prenom;
    private string service;
    private string mdp;
    private string login;


    2 références
    public Utilisateurs(int idUser, string nom, string prenom, string service, string mdp, string login)
    {
        this.idUser = idUser;
        this.nom = nom ?? string.Empty;
        this.prenom = prenom ?? string.Empty;
        this.service = service ?? string.Empty;
        this.mdp = mdp ?? string.Empty;
        this.login = login ?? string.Empty;
    }

    1 référence
    public int IdUser { get => idUser; set => idUser = value; }
    1 référence
    public string Nom { get => nom; set => nom = value; }

    1 référence
    public string Prenom { get => prenom; set => prenom = value; }

    2 références
    public string Service { get => service; set => service = value; }

    1 référence
    public string Mdp { get => mdp; set => mdp = value; }

    2 références
    public string Login { get => login; set => login = value; }
```

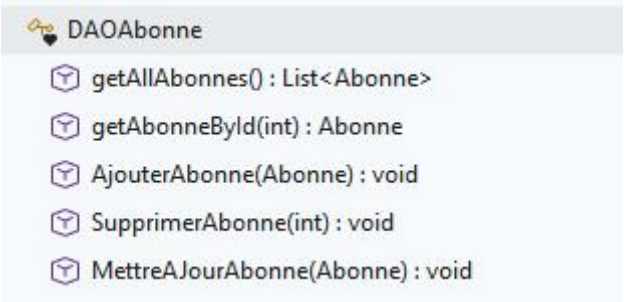
Couche donner DAO

Connexion a la base de donnée

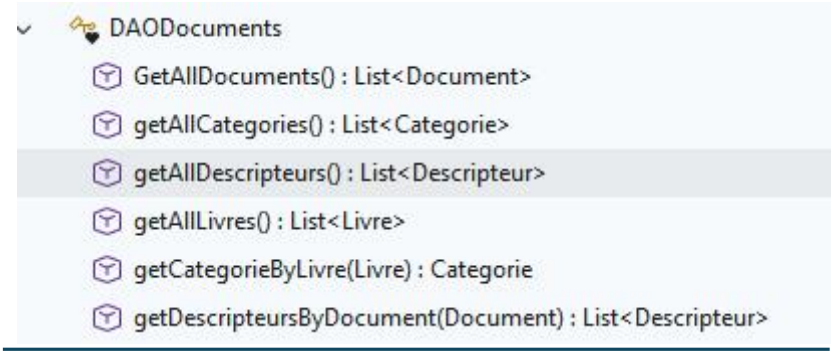
```
class DaoFactory {
private static MySqlConnection connection;
// Méthodes de gestion de connexion
public static void creerConnection();
public static void connecter ();
public static void deconnecter();
public static MySqlConnection GetConnection();
public static MySqlDataReader execSQLRead(string req);
```

Classe DAO & méthodes utiliser

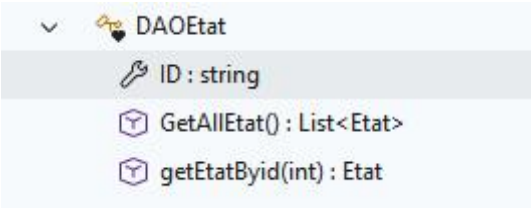
Class DAOAbonner



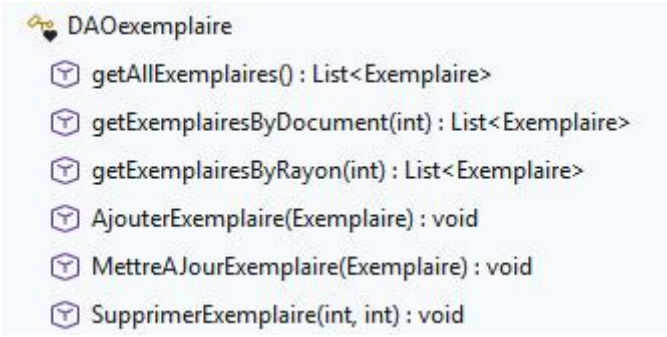
Class DAO Document



Classe DAOEtat



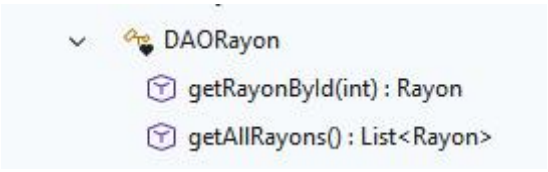
Classe DAOExemplaire



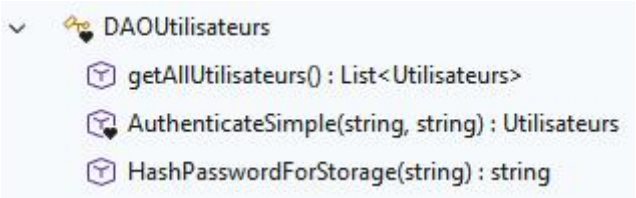
Classe DAOPresse



Classe DAORayon

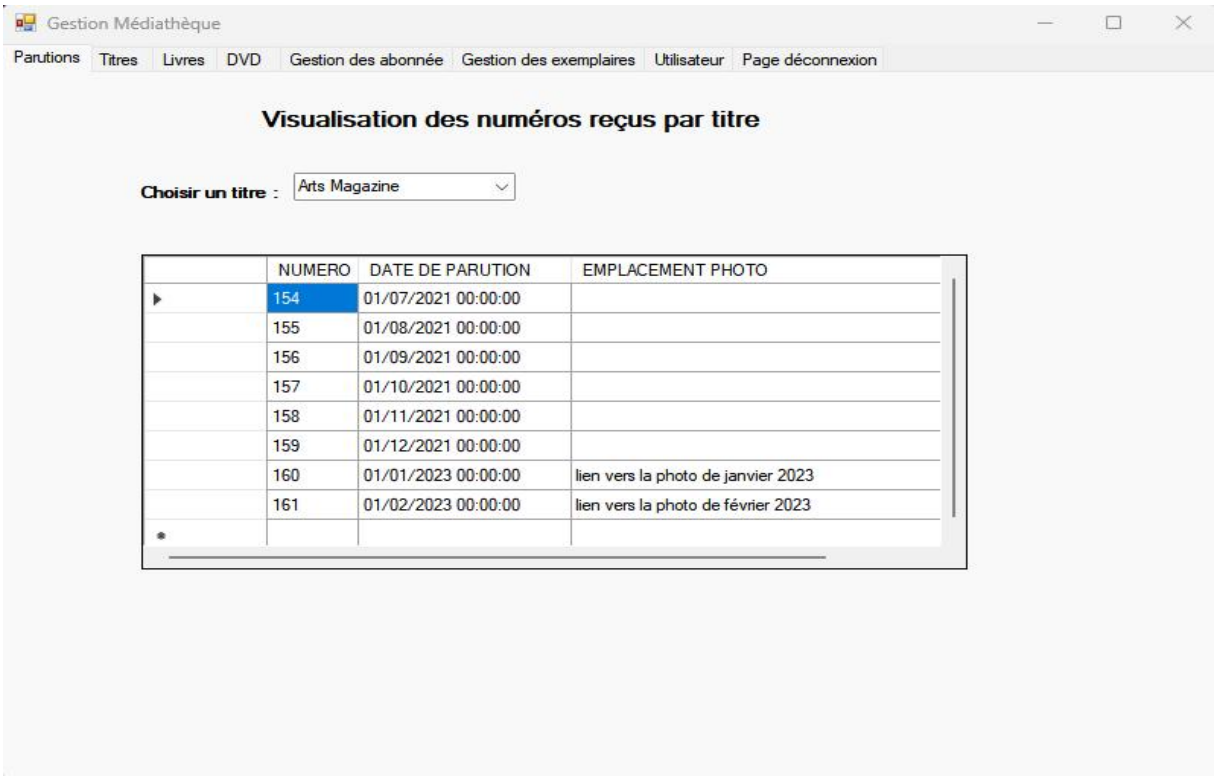


Classe DAOUtilisateur

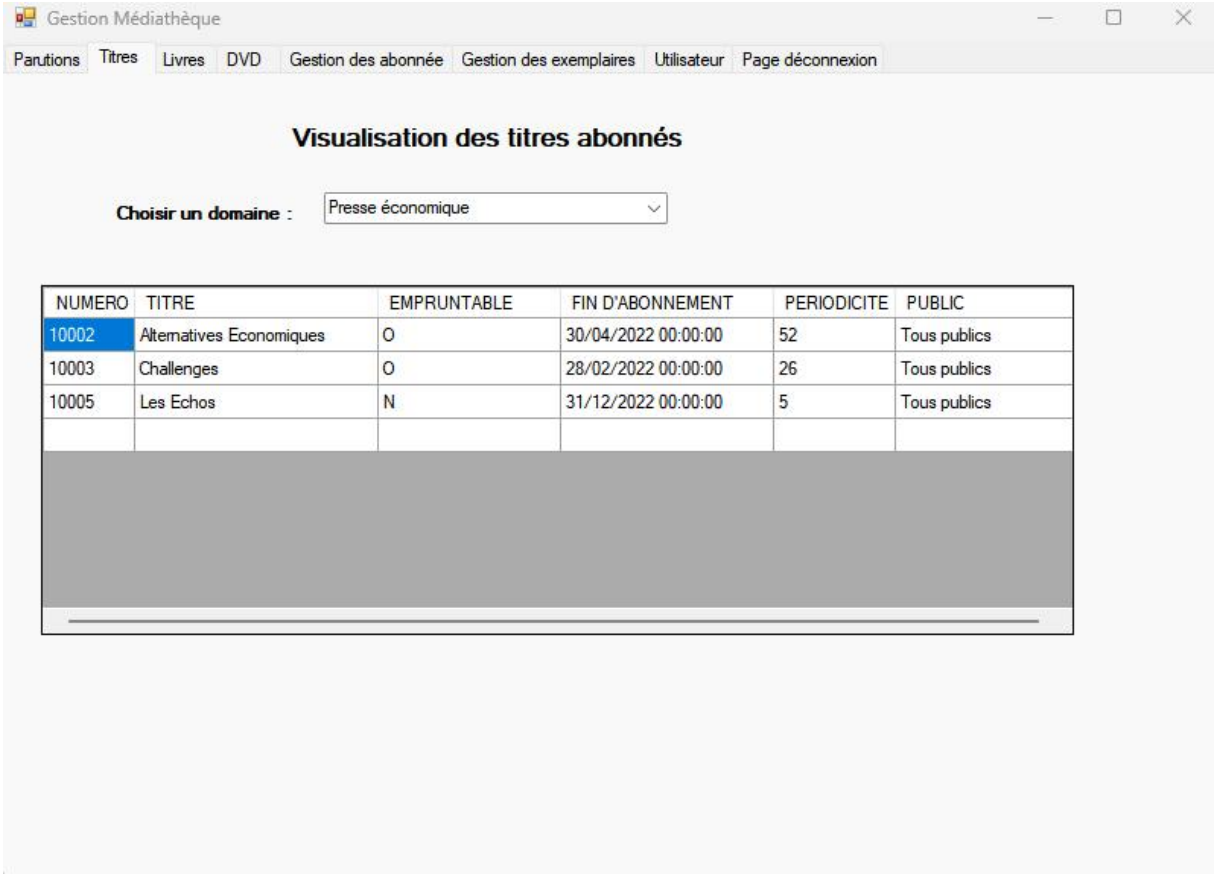


Interface utilisateur

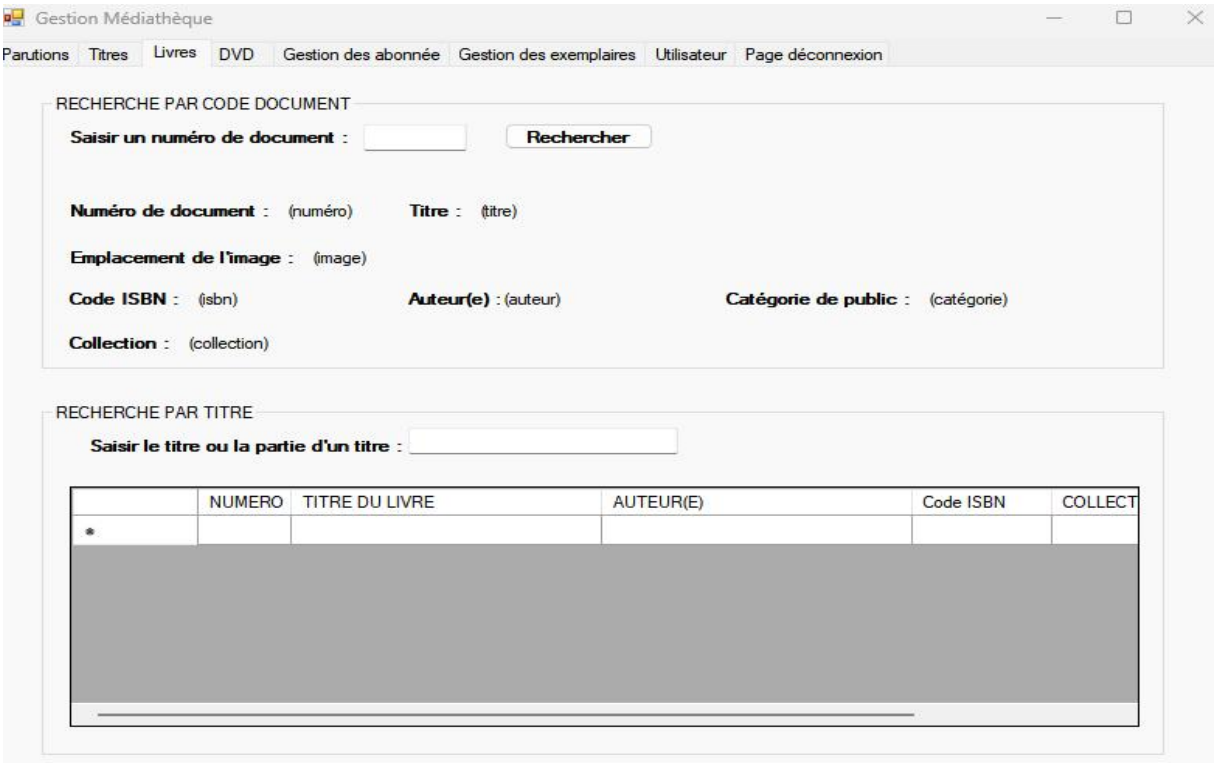
Onglet 1 Parutions : Gestion des parutions de revues



Onglet 2 Titres : Gestion des titres de revues

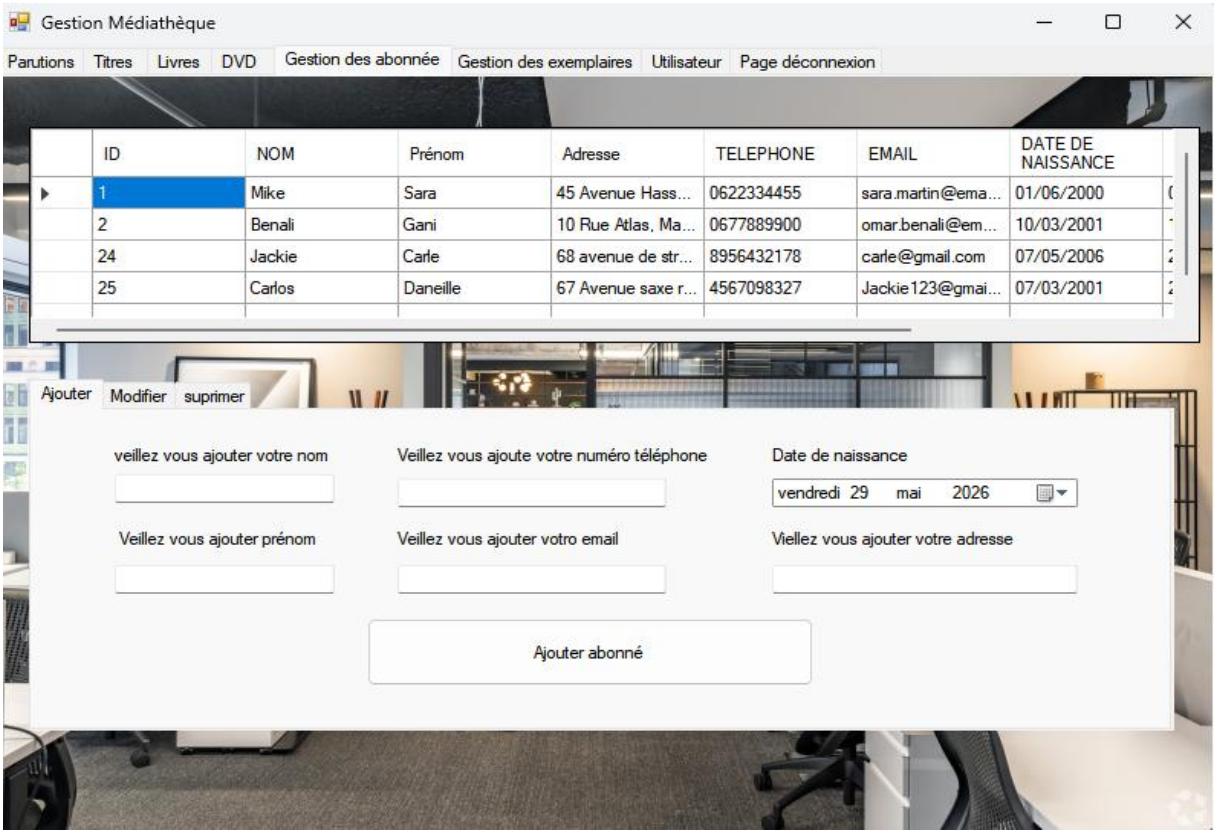


Onglet 3 Livres : Recherche et affichage des livres

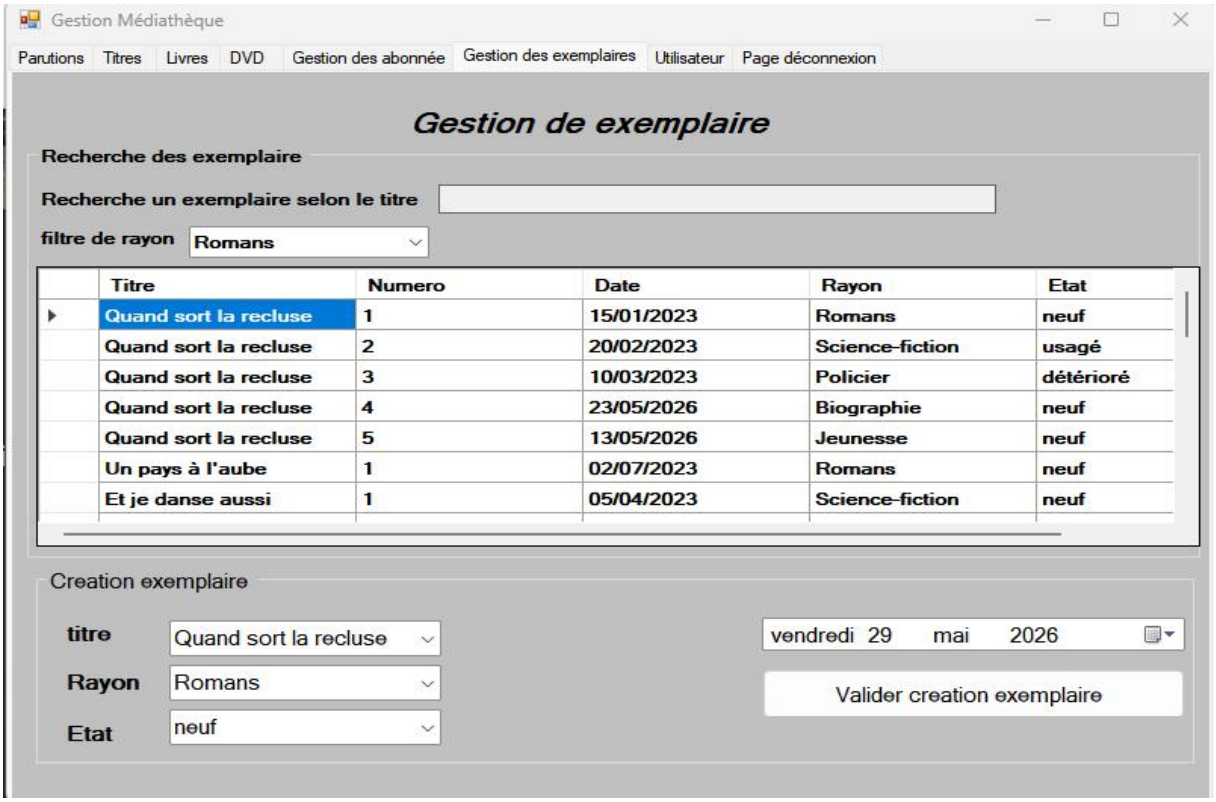


Onglet 5 CRUD Abonnés : Gestion des abonnés (Création, Lecture, Mise à jour,

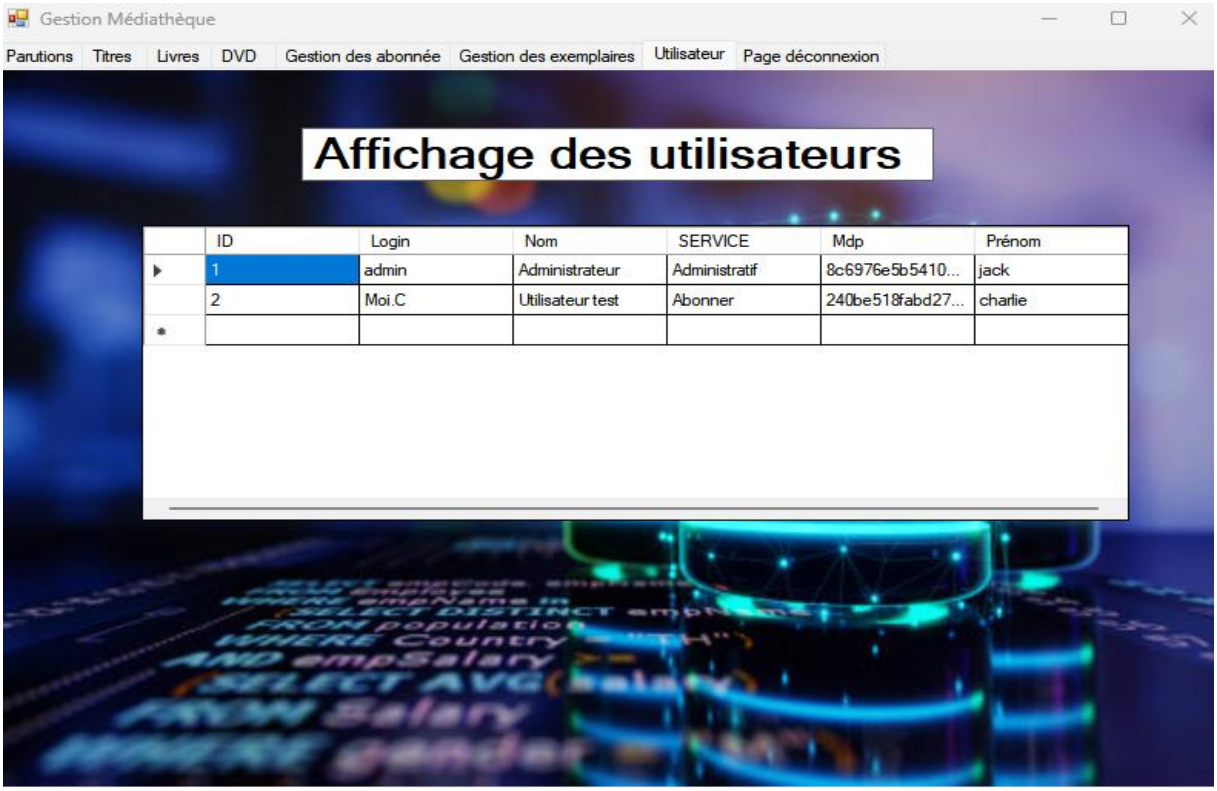
Suppression)



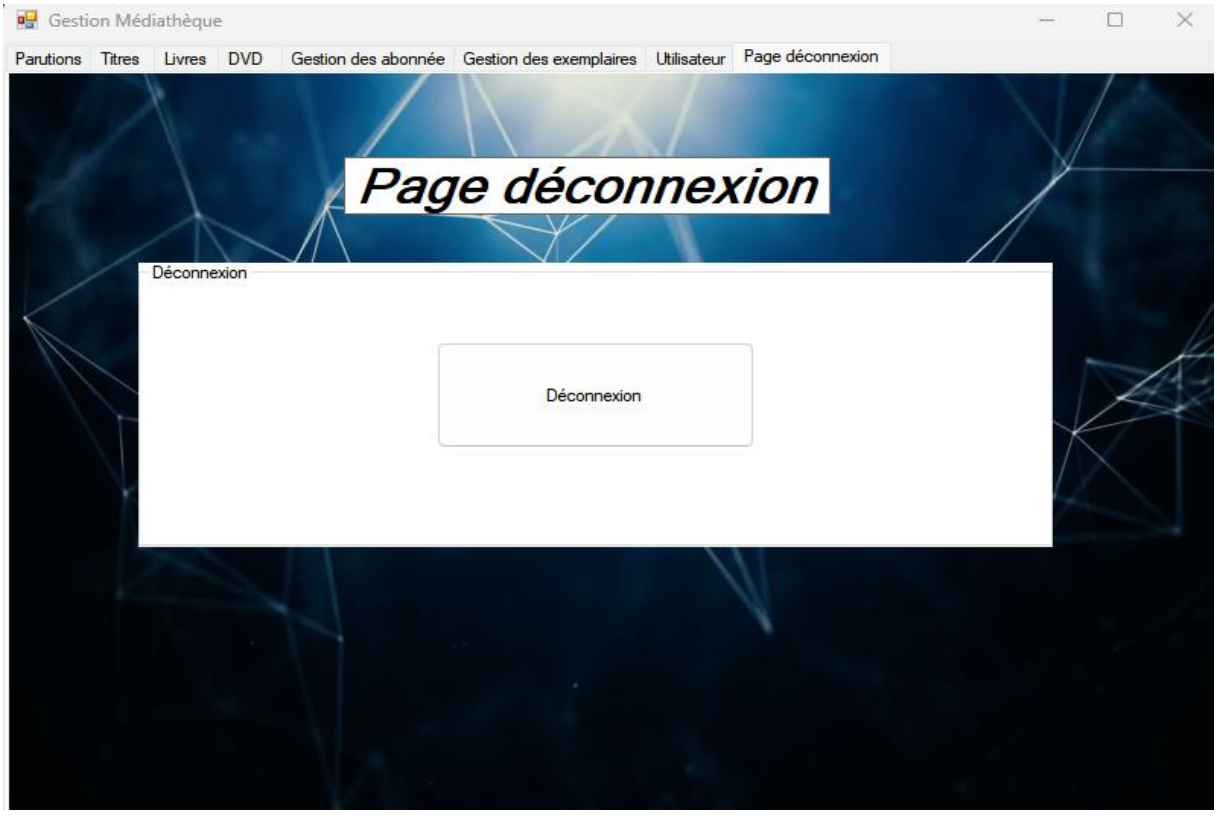
Onglet 6 exemplaires : affichage, recherche, filtrage et création



Onglet 7 Utilisateurs : Affichage des utilisateurs du système



Onglet 8 Gestion de la déconnexion de l'application



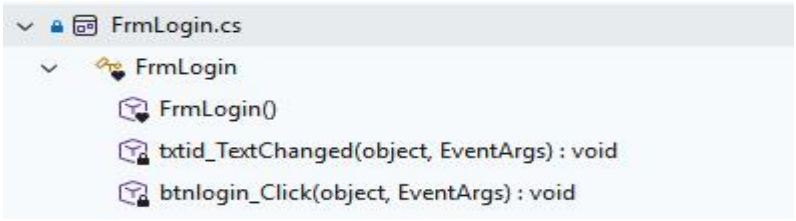
Form login

Méthodes d'événements utilisées

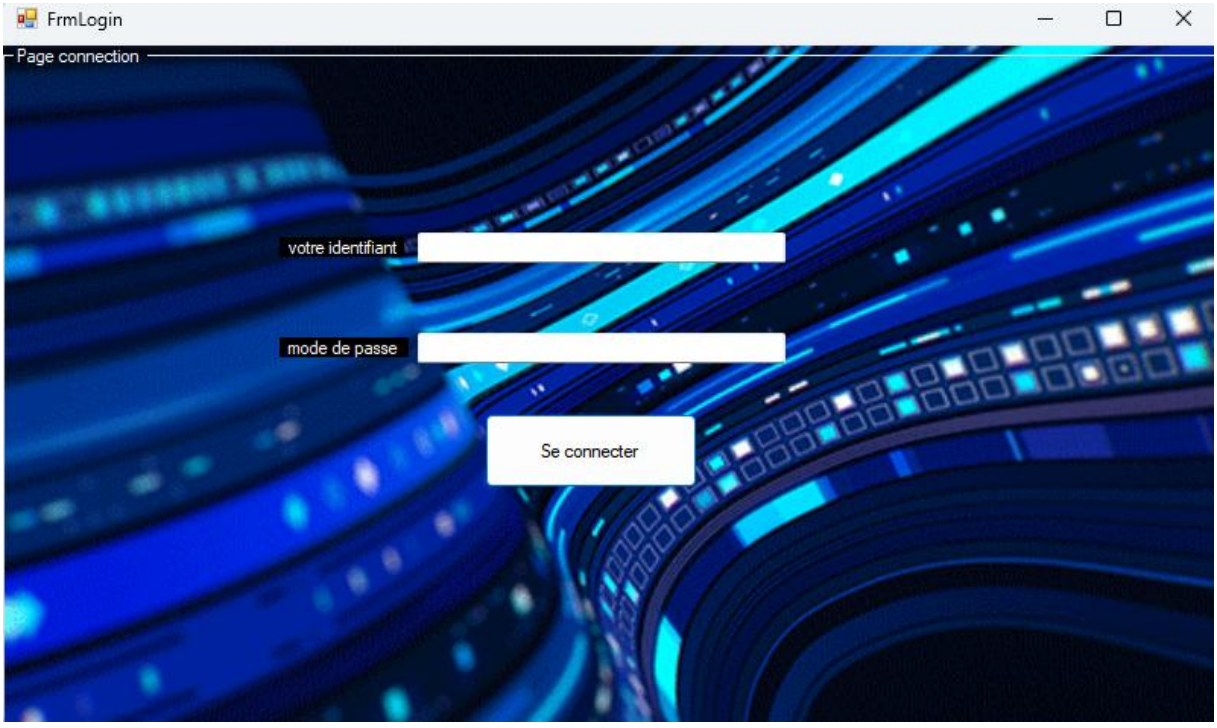
- Dans la classe FrmLogin, plusieurs méthodes d'événements sont mises en place pour gérer les interactions utilisateur :

- txtid_TextChanged(object sender, EventArgs e)
- Cette méthode est déclenchée à chaque modification du contenu du champ de saisie (TextBox). Elle permet de réagir dynamiquement aux changements effectués par l'utilisateur, par exemple pour contrôler ou valider la saisie.

- btnlogin_Click(object sender, EventArgs e)
- Cette méthode est appelée lorsque l'utilisateur clique sur le bouton de connexion. Elle contient généralement la logique d'authentification, comme la vérification des identifiants et l'accès à l'application.



L'interface de connexion est développée sous forme d'un formulaire Windows Forms nommé FrmLogin. Elle assure l'authentification de l'utilisateur via la saisie de ses identifiants. Ce formulaire est isolé des autres composants de l'application afin de gérer uniquement la logique de connexion



La méthode btnlogin_Click est une méthode événementielle déclenchée lors du clic sur le bouton de connexion, dont le rôle est de gérer le processus d'authentification de l'utilisateur : elle commence par récupérer les valeurs saisies dans les champs identifiant (txtid.Text) et mot de passe (txtmdp.Text), puis appelle la méthode AuthenticateSimple de la couche d'accès aux données (DAOUtilisateurs) afin de vérifier les informations dans la base de données ; avant cela, une validation est effectuée pour s'assurer que les champs ne sont pas vides, sinon un message d'erreur est affiché, et l'exécution s'arrête ; si un utilisateur valide est retourné (non nul), les informations de session sont enregistrées (Session.CurrentUser et Session.CurrentUserService), un message de bienvenue est affiché, puis le formulaire se ferme en renvoyant un résultat DialogResult.OK pour indiquer un succès ; dans le cas contraire, un message informe l'utilisateur que les identifiants sont incorrects, assurant ainsi la gestion complète des cas de réussite et d'échec de l'authentification.

```
1 référence
private void btnlogin_Click(object sender, EventArgs e)
{

    string login = txtid.Text;
    string password = txtmdp.Text;

    Utilisateurs utilisateur = DAOUtilisateurs.AuthenticateSimple(login, password);

    if (string.IsNullOrEmpty(login) || string.IsNullOrEmpty(password))
    {
        MessageBox.Show("Veuillez saisir votre identifiant et mot de passe.");
        return;
    }


    if (utilisateur != null)
    {
        // Authentification réussie : stocker la session (login + service)
        Session.CurrentUser = utilisateur.Login;
        Session.CurrentUserService = utilisateur.Service; // <- ajout

        MessageBox.Show("Bienvenue " + Session.CurrentUser + " du service " + Session.CurrentUserService);


        // Ferme la fenêtre de login et retourne DialogResult.OK à Program.Main
        this.DialogResult = DialogResult.OK;
        this.Close();


    }
    else
    {
        // Échec de l'authentification
        MessageBox.Show("Identifiant ou mot de passe incorrect.");

    }
```

Classe Session pour la gestion de l'authentification des utilisateurs

La classe Session est une classe statique utilisée pour gérer les informations de l'utilisateur actuellement connecté au sein de l'application. Elle contient des propriétés statiques telles que CurrentUser, qui stocke le login de l'utilisateur, et CurrentUserService, qui représente son rôle ou service, sans jamais enregistrer de données sensibles comme le mot de passe. La classe fournit également des propriétés calculées comme IsAuthenticated, permettant de vérifier si un utilisateur est connecté en testant si le login est renseigné, ainsi que IsAdmin, qui détermine si l'utilisateur possède des droits administrateurs en comparant son service à

la valeur "Administratif". Enfin, une méthode Logout() est implémentée afin de réinitialiser la session en mettant à null les informations utilisateur, permettant ainsi de gérer proprement la déconnexion et de sécuriser l'accès aux fonctionnalités de l'application

```
8 références
public static class Session
{
    // Login de l'utilisateur connecté
    4 références
    public static string CurrentUser { get; set; }

    // Service/rôle de l'utilisateur (ne pas stocker le mot de passe !)
    5 références
    public static string CurrentUserService { get; set; }

    // Vérification de l'authentification
    0 références
    public static bool IsAuthenticated => !string.IsNullOrEmpty(CurrentUser);

    // Vérification des droits administrateur
    2 références
    public static bool IsAdmin => string.Equals(CurrentUserService, "Administratif", StringComparison.OrdinalIgnoreCase);

    /// <summary>
    /// Déconnecte l'utilisateur actuel en vidant les données de session.
    /// </summary>
    1 référence
    public static void Logout()
    {
        CurrentUser = null;
        CurrentUserService = null;
    }
}
```

Evènement principaux

// Chargement initial d'application
private void FrmMediateq_Load(object sender, EventArgs e) {
// Connexion à la base de données
// Chargement des objets communs en mémoire
// Initialisation des contrôles
}

Fonctionnalité Gestion abonne

Fonction d'ajout d'un abonné dans la base de données via le bouton

La méthode btnAjouter_Click est une méthode événementielle déclenchée lors du clic sur le bouton d'ajout d'un abonné, dont l'objectif est de gérer l'insertion d'un nouvel abonné dans le système. Elle commence par vérifier que tous les champs obligatoires (nom, prénom, adresse, téléphone, email) sont correctement renseignés à l'aide de string.IsNullOrWhiteSpace, afin d'éviter toute saisie incomplète ; en cas de champs manquants, un message d'alerte s'affiche et le traitement est interrompu. Ensuite, elle initialise automatiquement les dates d'abonnement (date de début à la date actuelle et date de fin à +1 an), puis crée un objet Abonne avec les données saisies. Une validation complémentaire est réalisée via la méthode VerifierChamp() pour s'assurer que les données respectent les contraintes définies, suivie d'un contrôle de l'âge (l'utilisateur doit avoir au moins 18 ans). Si toutes les conditions sont respectées, l'abonné est enregistré dans la base de données via la méthode DAOAbonne.AjouterAbonne, un message de confirmation est

affiché, et la liste des abonnés est actualisée avec chargerabonnements(). Enfin, les champs de saisie sont réinitialisés pour permettre un nouvel ajout. En cas d'erreur technique, une exception spécifique est interceptée **Méthode gestion abonner ajouter**

```
// Évènement du bouton "Ajouter" de l'onglet Abonnements
1 référence
private void btnAjouter_Click(object sender, EventArgs e)
{
    try
    {

        if (string.IsNullOrWhiteSpace(txtNomajouter.Text) ||
        string.IsNullOrWhiteSpace(txtPrenomAjoute.Text) ||
        string.IsNullOrWhiteSpace(txtAdresse.Text) ||
        string.IsNullOrWhiteSpace(txtAjoutertelephone.Text) ||
        string.IsNullOrWhiteSpace(txtajouteremail.Text))
        {
            MessageBox.Show("Veuillez saisir tous les champs nécessaires.", "Champs manquants", MessageBoxButtons.OK, MessageBoxIcon.Warning);
            return;
        }

        DateTime datePremierAbonnement = DateTime.Now;
        DateTime dateFinAbonnement = datePremierAbonnement.AddYears(1);

        Abonne nouvelAbonne = new Abonne
        (
            0,
            txtNomajouter.Text,
            txtPrenomAjoute.Text,
            txtAdresse.Text,
            txtAjoutertelephone.Text,
            txtajouteremail.Text,
            dateTimeAjouteAbonne.Value,
            datePremierAbonnement,
            dateFinAbonnement

        );

        if (!VerifierChamp())
        {
            MessageBox.Show("Veuillez corriger les champs invalides avant d'ajouter l'abonné.", "Champs invalides",
```

```
    );

    if (!VerifierChamp())
    {
        MessageBox.Show("Veuillez corriger les champs invalides avant d'ajouter l'abonné.", "Champs invalides",
        MessageBoxButtons.OK, MessageBoxIcon.Warning);
        return;
    }

    if (dateTimeAjouteAbonne.Value > DateTime.Now.AddYears(-18))
    {
        MessageBox.Show("Vous devez avoir au moins 18 ans", "Attention", MessageBoxButtons.OK, MessageBoxIcon.Warning);
        return;
    }


    modele.DAOAbonne.AjouterAbonne(nouvelAbonne);
    MessageBox.Show("Abonné ajouté avec succès !", "Succès", MessageBoxButtons.OK, MessageBoxIcon.Information);
    chargerabonnements();
    // Réinitialisation des champs de saisie
    txtNomajouter.Text = "";
    txtPrenomAjoute.Text = "";
    txtajouteremail.Text = "";
    txtAjoutertelephone.Text = "";
    txtNomajouter.Text = "";
    txtAdresse.Text = "";

}
catch (ExceptionSIO exc)
{
    MessageBox.Show("Erreur lors de l'ajout de l'abonné : " + exc.Message, "Erreur", MessageBoxButtons.OK, MessageBoxIcon.Error);
}

/ Évènement du bouton "Supprimer" de l'onglet Abonnements
```

Méthode qui gère la suppression dans la base de données

```
1 référence
private void controlbtnSup_Click(object sender, EventArgs e)
{
    try
    {
        if (dataGrigestionAbonne.SelectedRows.Count > 0)
        {
            int selecteId = Convert.ToInt32(dataGrigestionAbonne.SelectedRows[0].Cells[0].Value);
            modele.DAOAbonne.SupprimerAbonne(selecteId);
            DialogResult result = MessageBox.Show("Êtes-vous sûr de vouloir supprimer cet abonné ?", "Confirmation de suppression",
            MessageBoxButtons.YesNo, MessageBoxIcon.Warning);
            chargerabonnements();


        }
        else
        {
            MessageBox.Show("Veuillez sélectionner un abonné à supprimer.", "Avertissement", MessageBoxButtons.OK, MessageBoxIcon.Warning);
        }

    }
    catch (ExceptionSIO exc)
    {
        MessageBox.Show("Erreur lors de la suppression de l'abonné : " + exc.Message, "Erreur", MessageBoxButtons.OK, MessageBoxIcon.Error);
    }
}
```

Méthode permettant la modification

```
1 référence
private void btnmodifierabonne_Click(object sender, EventArgs e)
{

    try
    {
        if (dataGrigestionAbonne.SelectedRows == null || dataGrigestionAbonne.SelectedRows.Count == 0)
        {
            MessageBox.Show("Veuillez sélectionner un abonné à modifier.", "Avertissement", MessageBoxButtons.OK, MessageBoxIcon.Warning);
            return;
        }

        var row = dataGrigestionAbonne.SelectedRows[0];


        int id = Convert.ToInt32(row.Cells[0].Value);


        string nom = txtmodifierNom.Text;
        if (nom == "") nom = row.Cells[1].Value.ToString();

        string prenom = txtmodifPrenom.Text;
        if (prenom == "") prenom = row.Cells[2].Value.ToString();

        string adresse = txtmodifadresse.Text;
        if (adresse == "") adresse = row.Cells[3].Value.ToString();

        string telephone = txtmodiftelephone.Text;
        if (telephone == "") telephone = row.Cells[4].Value.ToString();

        string email = txtmodifEmail.Text;
        if (email == "") email = row.Cells[5].Value.ToString();


        // Validation des champs obligatoires
        if (string.IsNullOrWhiteSpace(nom) || string.IsNullOrWhiteSpace(prenom))
        {
            MessageBox.Show("Le nom et le prénom sont obligatoires.", "Champs manquants", MessageBoxButtons.OK, MessageBoxIcon.Warning);
            return;
        }

        DateTime dateNaissance = DateTime.MinValue;
        DateTime datePremier = DateTime.MinValue;
        DateTime dateFin = DateTime.MinValue;
```

```
DateTime datePremier = DateTime.MinValue;
DateTime dateFin = DateTime.MinValue;
DateTime.TryParse(row.Cells[6].Value.ToString(), out dateNaissance);
DateTime.TryParse(row.Cells[7].Value.ToString(), out datePremier);
DateTime.TryParse(row.Cells[8].Value.ToString(), out dateFin);


if (dateTimeAjouteAbonne != null && dateTimeAjouteAbonne.Value != default(DateTime))
{
    dateNaissance = dateTimeAjouteAbonne.Value.Date;
}

// Validation de la date de naissance
if (dateNaissance == default(DateTime))
{
    MessageBox.Show("Veuillez saisir une date de naissance valide.", "Date invalide", MessageBoxButtons.OK, MessageBoxIcon.Warning);
    return;
}

if (datePremier == default(DateTime)) datePremier = DateTime.Now;
if (dateFin == default(DateTime)) dateFin = datePremier.AddYears(1);


Abonne abonne = new Abonne(id,nom,prenom,adresse,telephone,email,datePremier,dateFin,dateNaissance);


modele.DAOAbonne.MettreAJourAbonne(abonne);

MessageBox.Show("Abonné modifié avec succès.", "Succès", MessageBoxButtons.OK, MessageBoxIcon.Information);
chargerabonnements();

txtmodifierNom.Text = "";
txtmodifPrenom.Text = "";
txtmodifadresse.Text = "";
txtmodiftelephone.Text = "";
txtmodifEmail.Text = "";
dateTimeAjouteAbonne.Value = DateTime.Now;
```

Lig. : 528, Car. : 18

```
            dateTimeAjouteAbonne.Value = DateTime.Now;


        }
        catch (ExceptionSIO exc)
        {
            MessageBox.Show("Erreur SIO : " + exc.Message, "Erreur", MessageBoxButtons.OK, MessageBoxIcon.Error);
        }
        catch (Exception ex)
        {
            MessageBox.Show("Erreur : " + ex.Message, "Erreur", MessageBoxButtons.OK, MessageBoxIcon.Error);
        }

    }
```

Validation par expression régulière

La méthode VerifierChamp() est une méthode de validation qui contrôle les données saisies par l'utilisateur en vérifiant successivement le format de l'email et du numéro de téléphone à l'aide d'expressions régulières. Si l'un des champs est invalide, un message d'avertissement s'affiche et la méthode retourne false, sinon elle retourne true, autorisant ainsi le traitement des données.

public bool VerifierChamp()

{
    string patternEmail = @"^[^@\s]+@[^@\s]+\.[^@\s]+$";
    if (!Regex.IsMatch(txtajouteremail.Text, patternEmail))
    {
        MessageBox.Show("Veuillez saisir une adresse email valide.", "Format d'email invalide", MessageBoxButtons.OK, MessageBoxIcon.Warning);
        return false;

```
    }

    string patternTéléphone = @"^\d{10}$";
    if (!Regex.IsMatch(txtAjoutertelephone.Text, patternTéléphone))
    {
        MessageBox.Show("Veuillez saisir un numéro de téléphone valide (10 chiffres).",
"Format de téléphone invalide",
        MessageBoxButtons.OK, MessageBoxIcon.Warning);
        return false;
    }
    return true;

}

//Même concept pour le champ modification

public bool verifierChampModification()
{
    string patternEmail = @"^[^@\s]+@[^@\s]+\.[^@\s]+$";
    if (!Regex.IsMatch(txtmodifEmail.Text, patternEmail))
    {
        MessageBox.Show("Veuillez saisir une adresse email valide.", "Format d'email invalide",
MessageBoxButtons.OK, MessageBoxIcon.Warning);
        return false;
    }
    string patternTelephone = @"^\d{10}$";
    if (!Regex.IsMatch(txtmodiftelephone.Text, patternTelephone))
    {
        MessageBox.Show("Veuillez saisir un numéro de téléphone valide (10 chiffres).",
"Format de téléphone invalide",
        MessageBoxButtons.OK, MessageBoxIcon.Warning);
        return false;
    }
    return true;
}
```

Gestion exemplaire

La méthode ChargerExemplaires() récupère tous les exemplaires depuis la base de données via getAllExemplaires(), vide le tableau d'affichage, puis parcourt chaque exemplaire pour l'afficher dans le dataGridExemplaire avec son titre de document, numéro, date d'achat, rayon et état. En cas d'erreur, un message est affiché à l'utilisateur

```
2 références
public void ChargerExemplaires()
{
    try
    {


        lesExemplaires = modele.DAOexemplaire.getAllExemplaires();
        dataGridExemplaire.Rows.Clear();

        foreach (Exemplaire ex in lesExemplaires)
        {

            dataGridExemplaire.Rows.Add(
               ((Document)lesDocuments.Find(d => d.IdDoc == ex.IdDocument)).Titre,
                ex.Numero,
                ex.DateAchat.ToShortDateString(),
                ((Rayon)lesRayons.Find(r => r.IdRayon == ex.IdRayon)).Libelle,
                ((Etat)lesEtat.Find(etat => etat.ID == ex.IdEtat)).Libelle
            );
        }
    }

    catch (Exception exc)
    {
        MessageBox.Show("Erreur lors du chargement des exemplaires : " + exc.Message, "Erreur", MessageBoxButtons.OK, MessageBoxIcon.Error);
    }


}
```

La méthode btn_Exemplaire_Click() est déclenchée lors du clic sur le bouton d'ajout. Elle récupère les valeurs saisies dans le formulaire (document, rayon, état, date d'achat), puis valide la date en vérifiant qu'elle correspond exactement à la date du jour (ni future, ni passée). Si la validation est correcte, un nouvel exemplaire est créé et enregistré en base de données via AjouterExemplaire(), un message de succès s'affiche et la liste est rechargée via ChargerExemplaires(). En cas d'erreur, un message d'exception est affiché à l'utilisateur.

```
1 référence
private void btn_Exemplaire_Click(object sender, EventArgs e)
{


    try
    {
        int idDocument = Convert.ToInt32(cbxDocument.SelectedValue);
        int idRayon = Convert.ToInt32(cbxRayon.SelectedValue);
        int idEtat = Convert.ToInt32(cbxEtat.SelectedValue);
        DateTime dateAchat = dateTimePickerAjouterEx.Value;

        if (dateAchat.Date > DateTime.Now.Date)
        {
            MessageBox.Show("La date ne peut pas dépasser la date d'aujourd'hui !", "Date invalide",
                MessageBoxButtons.OK, MessageBoxIcon.Warning);
            return;
        }

        if (dateAchat.Date < DateTime.Now.Date)
        {
            MessageBox.Show("La date est déjà passée ! Veuillez saisir la date d'aujourd'hui.", "Date déjà passée",
                MessageBoxButtons.OK, MessageBoxIcon.Error);
            return;
        }


        Exemplaire nouvelExemplaire = new Exemplaire(idDocument, 0, dateAchat, idRayon, idEtat);
        modele.DAOexemplaire.AjouterExemplaire(nouvelExemplaire);

        MessageBox.Show("Exemplaire ajouté avec succès !", "Succès",
            MessageBoxButtons.OK, MessageBoxIcon.Information);

        ChargerExemplaires();
    }
    catch (ExceptionSIO exc)
    {
        MessageBox.Show("Erreur : " + exc.Message, "Erreur",
            MessageBoxButtons.OK, MessageBoxIcon.Error);
    }

}
```

Méthode filtrage par rayon

```
private void cbxfiltreRayon_SelectedIndexChanged(object sender, EventArgs e)
{
    dataGridExemplaire.Rows.Clear();
    Rayon rayonSelecte = (Rayon)cbxfiltreRayon.SelectedItem;

    if (rayonSelecte != null)
    {
        MessageBoxButtons buttons = MessageBoxButtons.OK;
        MessageBoxIcon icon = MessageBoxIcon.Information;
    }

    foreach (Exemplaire ex in lesExemplaires)
    {
        if (rayonSelecte.IdRayon == 0 || ex.IdRayon == rayonSelecte.IdRayon)
        {
            dataGridExemplaire.Rows.Add(
                ((Document)lesDocuments.Find(d => d.IdDoc == ex.IdDocument)).Titre,
                ex.Numero,
                ex.DateAchat.ToShortDateString(),
                ((Rayon)lesRayons.Find(r => r.IdRayon == ex.IdRayon)).Libelle,
                ((Etat)lesEtat.Find(etat => etat.ID == ex.IdEtat)).Libelle
            );
        }
    }
```

Méthodes recherche titre document

```
// Méthode de recherche d'exemplaires par titre de document
 private void txbRecherchEx_TextChanged(object sender, EventArgs e)
{

    dataGridExemplaire.Rows.Clear();

    foreach (Exemplaire ex in lesExemplaires)
    {
        string saisieMinusculesE = txbRecherchEx.Text.ToLower();
        txbRecherchEx.Text.ToLower();
        string titreMinusculesEX;

        titreMinusculesEX = ((Document)lesDocuments.Find(d => d.IdDoc ==
ex.IdDocument)).Titre.ToLower();
        if (titreMinusculesEX.Contains(saisieMinusculesE))
        {
            dataGridExemplaire.Rows.Add(
                ((Document)lesDocuments.Find(d => d.IdDoc == ex.IdDocument)).Titre,
                ex.Numero,
                ex.DateAchat.ToShortDateString(),
                ((Rayon)lesRayons.Find(r => r.IdRayon == ex.IdRayon)).Libelle,
                ((Etat)lesEtat.Find(etat => etat.ID == ex.IdEtat)).Libelle
            );
        }
    }
}
```

Sécurité

- Authentification des utilisateurs
- Requête préparée
- Expression régulière
- Hachage des mots de passe (SHA256)
- Gestion des sessions